

12/29/2024

Task and Event Scheduler

Presented by Team Alpha

Sufi Ali Qutub Zohaib

Uzair Ahmed Solangi

Fatima Abbas Laghari

Course Name:

Data Structure Fall 2024

Instructor Name:

Sir Abbas Mehdi Khoso

Abstract:

The Task and Event Scheduler helps users easily manage their tasks and events. It allows adding, deleting, and tracking tasks, as well as setting priorities. With its simple features, it enhances time management and makes organizing work more efficient.

SUKKUR IBA UNIVERSITY

Contents

Brief overview of the project	2
Key objectives	2
Summary of outcomes	3
1. Introduction	4
Background and context	4
Problem statement	4
Objectives and Scope.....	4
Importance of the project:.....	5
2. System Requirements	5
Functional requirements.....	5
Non-functional requirements:.....	6
Hardware and software requirements	6
3. Implementation.....	7
Description of modules or components	7
Module 1: Dashboard.....	7
Module 2: Task Manager	8
Module 3: Event.....	9
Integration process	10
4. Results and Discussion.....	11
Key outcomes of the project	11
Comparison with Objectives:	11
Challenges faced and how they were overcome	12

Brief overview of the project

This project is designed to assist users in managing their time and schedules effectively. The application allows users to add tasks and events, prioritize them using time and tags, delete tasks, and view their schedules in a structured format. It offers a user-friendly environment with an interactive interface that simplifies task management. The project emphasizes efficiency, helping individuals stay organized and productive in their daily activities.

Key objectives

1. Dashboard Features:

- Provide users with a dashboard where they can easily add tasks, view upcoming events, and check due tasks categorized by their tags.

2. Task and Event Management:

- Enable users to mark tasks and events as "done" or delete them when no longer needed.

3. Task and Event Details:

- Provide users with detailed information for each task and event, including the opening date, closing date, name, and associated tags.

4. Dependent Tasks:

- Allow users to create dependent tasks linked to specific events, ensuring these events cannot be marked as "done" until all dependent tasks are completed.

5. Task Prioritization and Organization:

- Automatically prioritize tasks based on their due dates and assigned tags to help users focus on the most important activities.
- Display all tasks due on the same date within a single frame, with "Next" and "Previous" buttons to navigate forward and backward for additional tasks or dates.

6. Completion Indicators:

- Use a green checkmark to visually indicate completed tasks and events, providing a clear overview of progress.

7. Tag Searching:

- Allow users to retrieve all tasks associated with a specific priority tag by clicking on its corresponding tag button on the dashboard.

Summary of outcomes

The Task and Event Scheduler successfully meets its core objectives by providing a user-friendly platform for managing tasks and events efficiently. Users can easily add, prioritize, and track their tasks and events, view them by due dates, and navigate through their schedule. The integration of task dependencies ensures that events are not marked as "done" until all related tasks are completed, promoting better task management and organization.

The application's dashboard displays necessary information, such as the opening date, names, and tags for each task and event, offering users a clear overview of their schedule. Tasks due on the same date are grouped together in a single frame for easy reference, while the "Next" button allows users to move forward and view additional tasks and events.

Completion indicators in the form of green checkmarks visually represent progress, further enhancing the user's experience. Overall, the project demonstrates effective time management and organization features, significantly improving task and event handling for users.

1. Introduction

Background and context

In today's technological era, managing time has become essential for individuals to stay organized and productive. With multiple tasks each day and numerous events throughout the week, it can be difficult to keep track of them all. Therefore, an organized system to schedule and prioritize tasks is necessary. This need has led to the development of task and event management applications, which help individuals remain productive and organized. This project aims to create one such application: a Task and Event Scheduler that enables users to manage their time efficiently by keeping track of tasks and events in a centralized platform.

Problem statement

It can be challenging for a person to manage multiple tasks and events with different priorities and deadlines. Many people struggle with remembering due dates, prioritizing tasks, and organizing their work efficiently. Traditional methods such as to-do lists or basic calendars may not offer sufficient features, leading to missed deadlines, confusion, and poor time management. There is a need for an application that not only tracks tasks and events but also provides features like task prioritization, dependency management, and a user-friendly interface to enhance productivity.

Objectives and Scope

The objective of this project is to develop a Task and Event Scheduler that allows users to:

- Add, delete, and manage tasks and events.
- Prioritize tasks based on due dates and tags.
- View tasks and events with their opening and closing dates, names, and tags.
- Create dependent tasks that must be completed before certain events can be marked as "done."
- Provide a dashboard for users to track their progress, navigate tasks efficiently, and visually mark tasks as completed.

The scope of this project includes the development of a desktop application with a user-friendly interface, supporting basic task management features, task dependencies, and event scheduling. The application will not cover complex project management features, but instead focus on providing an easy-to-use solution for individuals.

Importance of the project:

This project is important because it addresses a common issue many people face in today's world: managing multiple tasks and deadlines can be overwhelming and difficult. By providing an organized way to handle tasks and events, this project aims to improve productivity, reduce stress, and help users stay on track with their schedules. The application's prioritization and dependency features add value by allowing users to focus on the most critical tasks first and ensuring that tasks are completed in the correct sequence. Ultimately, this project helps individuals and teams achieve better time management and enhance their overall efficiency.

2. System Requirements

Functional requirements

1. Task and Event Management:

- Users should be able to add new tasks and events.
- Users should be able to view a list of all tasks and events, with details such as opening date, closing date, name, and associated tags.
- Users should be able to mark tasks or events as "done" and delete them when necessary.

2. Task Prioritization:

- The system should prioritize tasks based on their due dates and tags.

3. Dependent Tasks:

- Users should be able to create dependent tasks, where specific tasks need to be completed before the event can be marked as "done."

4. Navigation:

- All tasks due on the same date should be shown in a single frame.
- A "Next" button should be available to allow users to move forward to view additional tasks or events for subsequent dates.

5. Completion Indicators:

- A green checkmark should appear when tasks or events are marked as "done."

6. Tag Searching:

- User should be able to get all tasks of same priority by clicking on the tag button of that priority.

Non-functional requirements:

1. Usability:

- The application should be easy to use, with a simple, user-friendly interface.

2. Performance:

- The system should perform efficiently, handling a reasonable number of tasks and events without noticeable delays.

3. Reliability:

- The system should be reliable, with no crashes or loss of data.

4. Scalability:

- The system should be able to handle an increasing number of tasks and events over time, allowing users to expand their usage as needed.

5. Security:

- User data, such as tasks and events, should be stored and managed securely using appropriate data structures, ensuring no unauthorized access.
- Access to sensitive information, like task details and event schedules, should be protected with proper handling within the application, minimizing risks of data loss or corruption.

Hardware and software requirements

• Hardware requirements:

1. **Processor:** Core i3 or above
2. **RAM:** 2GB
3. **Storage:** >500MB
4. **Display:** minimum 800x600 pixels (which can support java swing GUI)
5. **Input Device:** Keyboard & Mouse.

• Software requirements:

1. **OS:** Windows 7 or above
2. **JDK (JRE included):** JDK 8 (Java SE 8)
3. **IDE:** Visual Code, NetBeans or any.

3. Implementation

Description of modules or components

Module 1: Dashboard

Purpose:

The Dashboard serves as the central hub for the application, providing an easy-to-navigate interface for managing tasks and events.

Features:

- **Card Layout Structure:** The dashboard is designed with a card layout to display various features in an organized manner.
- **Tag Buttons (North):** “Important”, “Normal”, and “Urgent” buttons display tasks with matching tags, showing task names and due dates.
- **Navigation (South):** “Next” and “Previous” buttons to navigate through tasks at different dates.
- **Task Management (West):**
 - **Add Tasks:** Opens a frame for adding new tasks.
 - **Open Events:** Opens a frame for event management.

Functionality:

- Clicking tag buttons filters tasks by urgency.
- The "Next" and "Previous" buttons allow navigation through tasks.
- "Add Tasks" allows users to input task details, and "Open Events" manages events.

Interaction:

- Connects with Task and Event Management modules to retrieve and manage data.

Module 2: Task Manager

Purpose:

The Task Manager module handles task creation, prioritization, and completion, organizing tasks by tags for efficient management.

Features:

- **Task Creation:** Takes task name, date, and tag as input.
- **Task Prioritization:** Tasks are stored and prioritized using hash maps based on their tags (Important, Normal, Urgent).
- **Task Display:** On the Dashboard, tasks are shown with their name, due date, and options to delete or mark as done.
- **Task Class:** Attributes include task name, date, tag, and a flag (isCompleted) to track task completion.

Functionality:

- Tasks are categorized by tags and displayed on the Dashboard.
- Users can delete or mark tasks as complete through the Dashboard interface.

Interaction:

- Integrates with the Dashboard module to display tasks and update their status (completed or deleted).

Error Handling:

- Validates task input (e.g., missing fields).

Module 3: Event

Purpose:

The Event Manager module handles event creation, display, and management, allowing users to add tasks to events and track progress.

Features:

- **Layout:** Uses a null layout with setBounds to position components.
- **Event Display:** The module opens when the user clicks "Open Events" on the Dashboard.
 - **Clock:** Displays the current time at the top.
 - **Home Button:** Allows navigation back to the Dashboard.
 - **No Upcoming Events:** Displays a message if there are no upcoming events.
- **Event Creation:** A panel with input fields for adding event details (name, opening and closing date).
 - Once an event is added, another panel appears showing the event's opening and closing dates, along with a quote by Peter Drucker.
 - Options to add another event or add tasks to the event.
- **Task Management:** If "Add Task" is selected, a new panel prompts the user to add task details.
- **Task Linking:** Once added, the event is displayed on the Dashboard under the corresponding date.
 - Clicking on the event name shows event details and allows users to mark related tasks as completed. An event can't be marked done, if dependent tasks are not completed.

Functionality:

- Events can be added, and tasks can be linked to events.
- Tasks associated with events can be marked as done through the Dashboard.

Interaction:

1. Integrates with the Dashboard to display upcoming events and tasks, updating dynamically as events and tasks are added or completed.

Integration Process and Code Structure

1. Identification and integration of GUI Components to be integrated:

- Main Frame. (MainFrame.java)
- Dashboard panel. (DashBoard.java)
- Event panel. (OpenEvent.java)
- Add Task panel. (JOptionPane)
- Date Panel (DatePanel.java)

Main frame consists of one object of DashBoard.java and one of OpenEvent.java. Main frame uses **card layout to switch** between Dashboard and Event panel. **“Open Event” button** on Dashboard switch to Event panel. **“Home” button** on Dashboard switch to dashboard panel. **Action listeners** for both (using their respective object) are implemented in MainFrame.java.

“Add Task” button on Dashboard panel opens **JOptionPane** allowing user to enter detail of individual tasks.

Dashboard (Border Layout) **uses card layout** at its center position for dynamically adding date panel. Task/Events of same date/opening date are shown in one date panel.

2. Integration of Data Structures:

- Doubly Linked List (DoublyLinkedList.java) [is used in DashBoard.java]
- Hash Map (HashMap.java) [is used in MainFrame.java]
- Priority Queue (build in java.util.PrioritQueue) [is used in OpenEvent.java]
- Array List (build in java.util.ArrayList) [is used in DashBaord.java, DatePanel.java]

Doubly Linked List is used (with addition of **currentNode** attribute in implementation) to store the names for different **date panels cards** to be added at the center positon of dash board. Using currentNode the current date panel is tracked and next() and prev() methods are used to **navigate between date panels**.

One **static** object of **HashMap** is created in the Main Frame class. Whenever any task either in any event or individually at dashboard is added, **looking at its priority tag** is added into hash map.

Priority Queue is used in Event class to add tasks for Events according to priority tag. The task with **highest priority (important) is given least value** that's why PQ is **min heap**. User cannot mark any task as done without completing all its tasks.

Array List is used in DashBoard.java to add date panels and in DatePanel.java to add events and tasks dynamically. Tasks from Dashboard.java and events from OpenEvent.java are added to the Array list of DatePanel.java. **Static addEvent(Event e) method of DashBoard.java is called from OpenEvent.java to add events to dashboard dynamically.**

4. Results and Discussion

Key outcomes of the project

- ✓ **Task and Event Management:**
Users can add, delete, and manage tasks and events, allowing efficient tracking and organization.
- ✓ **Task Prioritization:**
Tasks can be prioritized based on due dates and urgency tags (Important, Normal, Urgent), ensuring that users focus on high-priority tasks.
- ✓ **Detailed View of Tasks and Events:**
Tasks and events are displayed with essential details such as names, due/opening/closing dates, and tags, enabling users to stay informed.
- ✓ **Dependent Task Functionality:**
Dependent tasks can be created, ensuring certain tasks must be completed before an event can be marked as "done."
- ✓ **Dashboard for Progress Tracking:**
A dashboard was implemented to help users track tasks and events, offering easy navigation and visual indicators to mark tasks as completed.

Comparison with Objectives:

Objective 1: Add, delete, and manage tasks and events

- **Outcome:** Achieved with the implementation of task and event management features, including adding and deleting tasks and events.

Objective 2: Prioritize tasks based on due dates and tags

- **Outcome:** Tasks are successfully prioritized using tags (Important, Normal, Urgent) and sorted based on due dates.

Objective 3: View tasks and events with essential details

- **Outcome:** Completed with a clear display of tasks and events, showing names, due/opening/closing dates, and tags.

Objective 4: Create dependent tasks

- **Outcome:** Dependent tasks were successfully integrated, ensuring that certain tasks must be marked complete before specific events can be marked "done."

Objective 5: Provide a dashboard to track progress

- **Outcome:** The dashboard was implemented as a central hub for tracking tasks, with navigation and visual indicators for task completion.

Challenges faced and how they were overcome

Challenge 1: Handling Task Prioritization

- **Solution:** Implemented hash maps to prioritize tasks based on tags (Important, Normal, Urgent). This enabled efficient categorization and retrieval of tasks according to their tags and due dates.

Challenge 2: Navigation of date panel

- **Solution:** Card names for date panel is **added sorted** in doubly linked list for traversal between different dates.

Challenge 3: Designing the Event Manager Interface

- **Solution:** The null layout structure was used for positioning components, and extensive testing ensured correct display of event and task panels across different screen sizes.

Challenge 4: Synchronizing Task and Event Updates

- **Solution:** Seamless integration was developed between the Task Manager and Event Manager to ensure that tasks linked to events were accurately reflected in the Dashboard in real-time.

Challenge 5: Implementing Event Prioritization with Priority Queue

- **Solution:** A Priority Queue class was implemented to manage event prioritization efficiently. This required overcoming challenges related to ensuring the queue correctly handled event priorities based on their importance or due dates, ensuring smooth scheduling and event management.

Challenge 6: User Input Validation

- **Solution:** Input validation was implemented for both tasks and events to ensure all required fields were filled and data integrity was maintained.